



西安电子科技大学
XIDIAN UNIVERSITY

《操作系统》

第五章 设备管理



人工智能学院

【考纲内容】

（一）输入/输出（I/O）管理基础

设备：设备的基本概念，设备的分类，I/O接口；**I/O控制方式**：轮询方式，中断方式，DMA方式；**I/O软件层次结构**：中断处理程序，驱动程序，设备独立性软件，用户层I/O软件；**输入/输出应用程序接口**：字符设备接口，块设备接口，网络设备接口，阻塞/非阻塞I/O

（二）设备独立软件

缓冲区管理；设备分配与回收；假脱机技术（SPOOLing）；设备驱动程序接口

（三）外存管理

磁盘：磁盘结构，格式化，分区，磁盘调度算法；**固态硬盘**：读/写性能特效，磨损均衡

【复习提示】

本章的内容较为分散，**重点掌握I/O接口、I/O软件、三种I/O控制方式、高速缓存与缓冲区、SPOOLing技术、磁盘特性和调度算法**。本章很多知识点与硬件高度相关，建议与计算机组成原理的对应章节结合复习。已复习过计算机组成原理的读者遇到比较熟悉的内容时也可适当跳过。

本章内容在历年统考真题中所占的比重不大，若统考中出现本章的题目，则基本上可以断定一定较为简单，看过相关内容的读者就一定会做，而未看过的读者基本上只能靠“蒙”。考研成功的秘诀是复习要反复多次且全面，偷工减料是要吃亏的，希望读者重视本章的内容。

一、引言

二、I/O系统概述

三、I/O软件的组成

四、具有通道的设备管理

五、缓冲技术

一、引言

设备的概念

- ❖ 一个计算机系统就是由大量的设备构成的，例如：**CPU**，磁盘，显卡、显示器、鼠标、键盘等。这些设备的特点和功能各不相同。在这些设备中，有一类是作为计算机系统与外界交互的工具使用的，它具体负责计算机与外部的输入输出(I/O)工作，我们称这类设备为**外部设备**，简称为**外设**，本章重点研究的就是操作系统中对这类设备的管理策略。

设备管理的目标

- ❖ 提高设备的利用率

就是提高**CPU**与**I/O**设备之间的并行操作程度。

- ❖ 为用户提供方便统一的界面

- 方便：是指用户能独立于具体设备的复杂物理特性之外而方便地使用设备；
- 统一：是指对不同的设备尽量使用统一的操作方式。

一、引言

I/O和设备管理连接硬件与上层功能，与其他模块关系紧密：

- **与进程管理：**进程的I/O请求触发设备管理的操作，进程会因I/O操作阻塞/唤醒；设备管理需协调多进程对I/O设备的竞争，保障进程并发执行
- **与存储管理：**I/O设备（如磁盘）是存储管理中外存的载体，存储管理分配内存缓冲区用于I/O数据暂存，虚拟内存的换入换出也依赖I/O设备操作
- **与文件管理：**文件的读写等操作，需通过I/O设备管理实现数据在存储设备（如磁盘）和内存间的传输，是文件访问的基础支撑

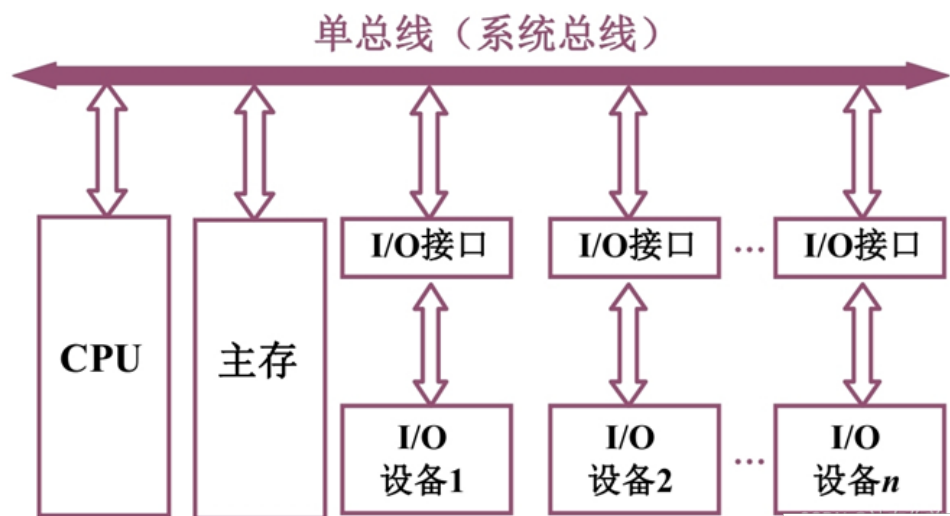
二、I/O系统概述

I/O系统的定义

- ❖ 计算机中负责管理I/O的机构称为**I/O系统**（硬件和软件的组合）。

I/O系统的结构

- ❖ 单总线结构



CPU、内存、各类外设（磁盘、打印机等），通过统一的**总线（bus）**连接，所有设备共享同一条数据传输通道。

CPU/内存：系统核心，负责运算、暂存数据；

外设（磁盘、打印机）：需通过控制器连到总线（比如磁盘控制器管理磁盘读写，打印机控制器处理打印指令）

控制器：是“翻译官”，把设备指令转成总线能理解的信号

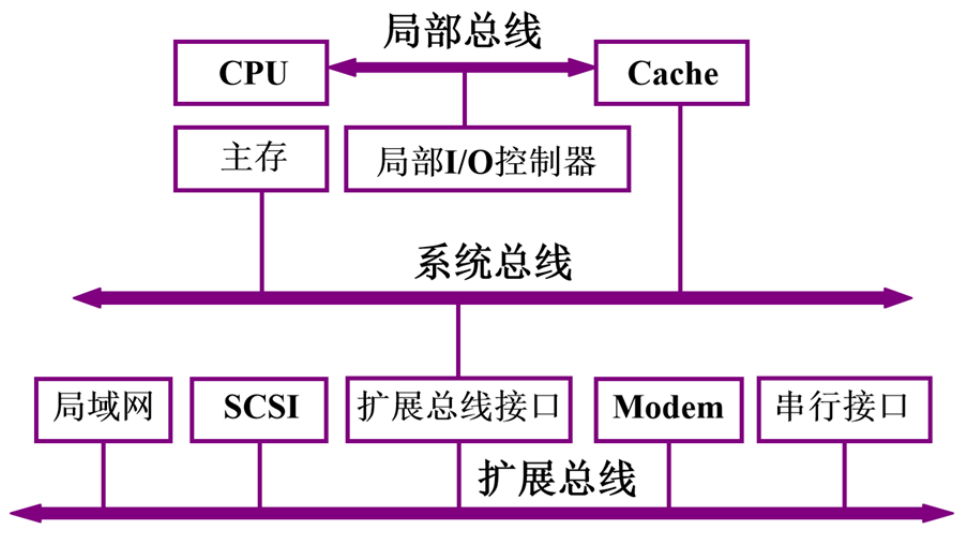
优点：结构简单，布线、设计成本低，适合早期简单计算机系统

缺点：所有设备抢同一条总线，数据传输易“堵车”（如CPU读内存时，打印机想发数据就得等），效率受限

二、I/O系统概述

I/O系统的结构

❖ 多总线结构



局部总线：连接最核心的部件（处理器、高速缓存、主存储器、本地I/O控制器），专用于高频、低延迟的数据交互（比如CPU与内存直接通信，走这条“**快车道**”）

系统总线：连接局部总线和其他扩展设备，承担中等速率的数据传输，是系统内部的“**主干道**”

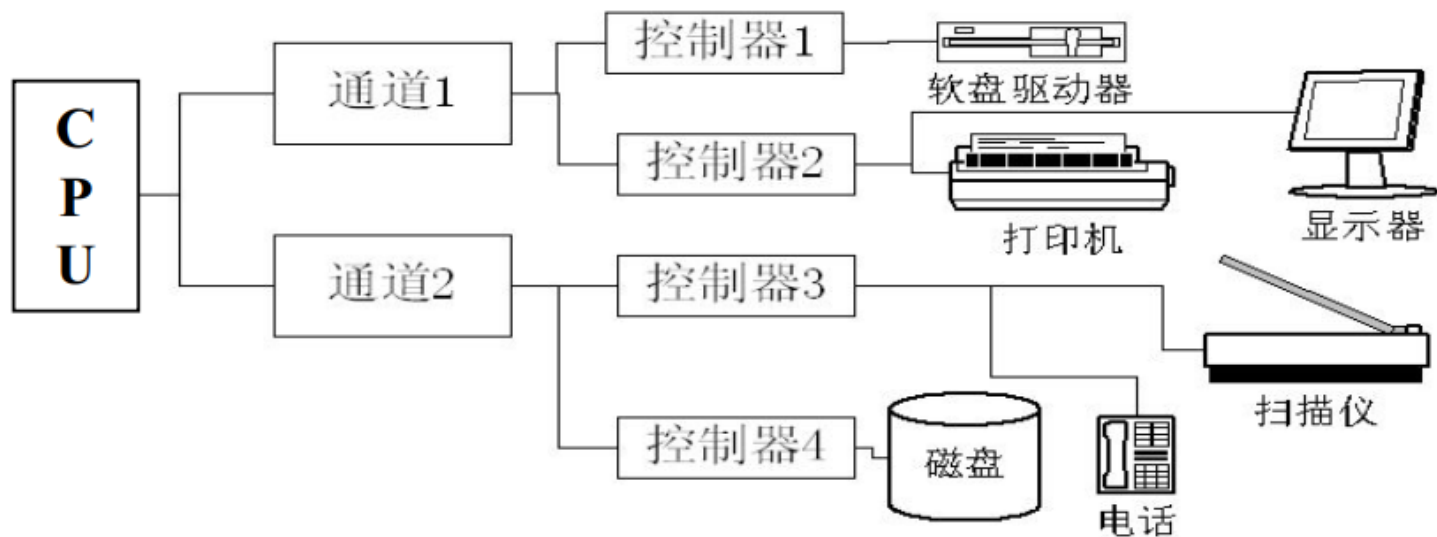
扩充总线：连接外围低速设备（网络、Modem、串行接口等），允许这些设备“**慢节奏**”收发数据，不影响核心总线的效率

优点：按设备速率分层，减少总线冲突，提升整体效率；支持更多设备扩展（通过扩充总线接各类外设），适配复杂场景

缺点：结构复杂，设计、维护成本高，需要协调多层总线的通信逻辑

二、I/O系统概述

❖ 具有通道系统的I/O系统

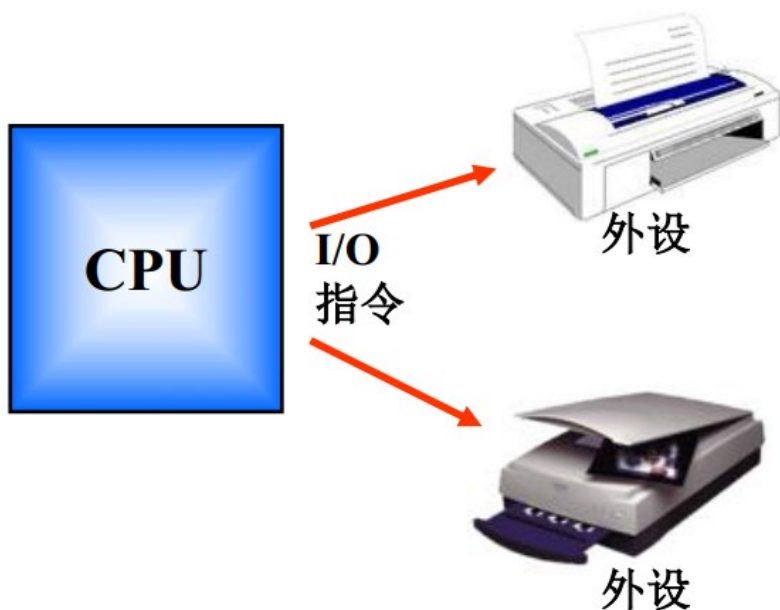


- **CPU**：系统运算核心，是I/O操作的“总指挥”
- **通道（通道1、通道2）**：CPU与外设间的“专属调度员”，负责接管I/O指令、协调设备通信，让CPU能专注运算，提升并行效率
- **控制器（控制器1-4）**：通道与具体外设的“中间人”，比如控制器1连软盘驱动器，控制器2连打印机，负责把通道指令转成设备能懂的操作
- **外设**：软盘驱动器、打印机、显示器、扫描仪、磁盘、电话等，是实际执行I/O动作（读写、显示、扫描）的硬件

二、I/O系统概述

I/O系统的控制方式

❖ 程序控制I/O（直接控制方式）



是一种基础的I/O控制逻辑：由CPU直接通过I/O指令操控外设（如打印机、扫描仪），全程需CPU主动参与、等待设备状态

➤ 从用户空间拷贝数据到内核buffer；逐字节发送数据时，CPU持续循环检测（while循环）设备状态寄存器，直到设备准备好接收；设备就绪后，才写入数据寄存器完成输出

```
Copy_from_user (buffer, p, count);
for (i=0; i<count; i++)
    while (*printer_status_reg!=READY);
    //循环检测设备是否就绪
    *printer_data_register=p[i];
    //设备就绪后，发送数据
Return_to_user();
```

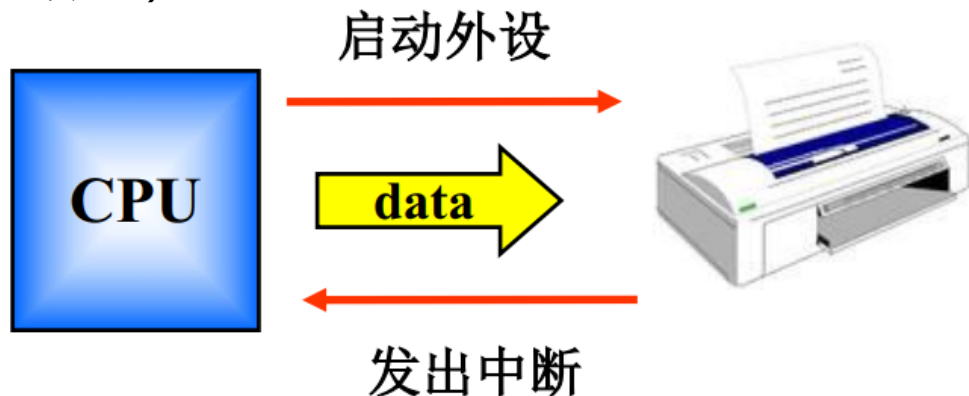
优点：逻辑简单，无需复杂硬件/软件协同，直接通过程序指令控制

缺点：CPU要反复检测设备状态，大量时间浪费在“等待、测试”，导致整体效率低，没法同时做其他运算任务

二、I/O系统概述

❖ 中断驱动I/O

CPU先启动外设并发送数据，之后不用“死等”设备完成；**外设处理完数据后，主动发出中断信号通知CPU**，CPU再响应中断做后续处理（比如确认传输完成、处理新数据）

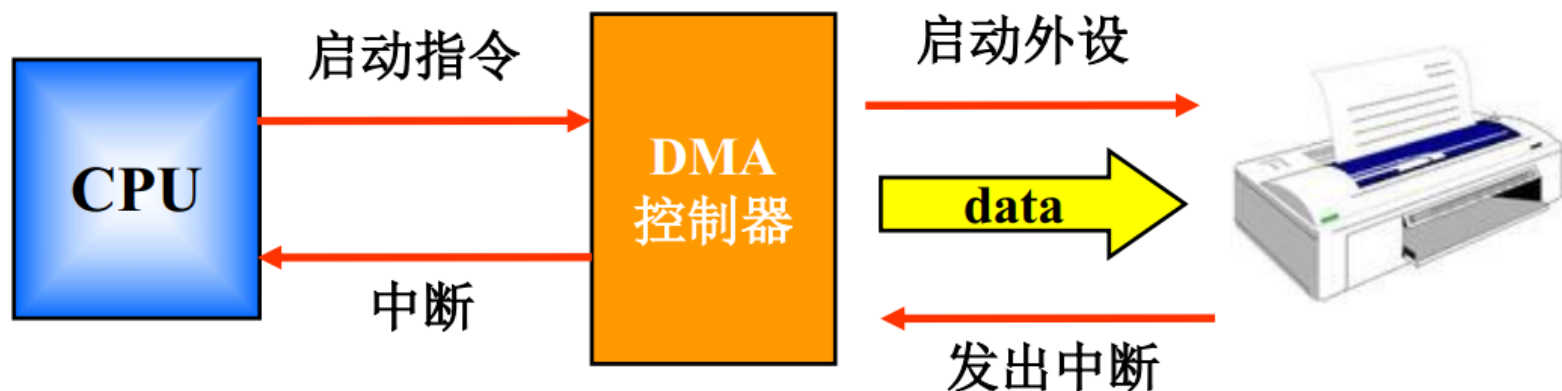


优点：让CPU摆脱“全程紧盯设备”的状态，外设处理数据时，CPU能去执行其他程序/任务（比如同时跑办公软件、后台进程），实现**CPU与外设并行工作**，大幅提升CPU利用率——这是操作系统“多任务并发”的基础支撑之一。

缺点：数据传输仍依赖CPU参与，且单次传输数据量小（得频繁触发中断），所以**只适合低速外设**（如打印机、键盘），面对高速设备（如磁盘、网卡）会因中断太频繁，反而拖慢系统。

二、I/O系统概述

❖ 直接存储访问I/O(DMA, Direct Memory Access)



CPU只需发启动指令给**DMA控制器**，指定数据传输的源、目的和长度；之后数据读写全程由DMA控制器接管（比如外设-内存直接传数据），CPU能去执行其他任务；传输完成后，DMA控制器发中断通知CPU收尾。

优点：彻底解放CPU——对比中断驱动I/O（数据传输仍需CPU参与），DMA让数据搬运和CPU运算完全并行——CPU只需管“开始和结束”，中间全程不插手，**适合高速设备**（如磁盘、网卡），能高效处理大数据量传输

缺点：缓存一致性问题——CPU常在高速缓存中保留内存数据的副本。当DMA控制器直接在内存中写入数据，而绕过CPU缓存时，就会导致缓存中的数据与内存中的实际数据不一致。

二、I/O系统概述

DMA从磁盘读数据到内存的流程——举例

1. 初始化与准备

- CPU 设置DMA控制器：
 - 目标内存起始地址
 - 要传输的数据量（字节数）
 - 磁盘的起始扇区地址

2. 发起I/O请求

- CPU 向磁盘控制器发出“读”命令，并将DMA信息告知它。
- CPU 将当前进程挂起，执行其他任务。

3. DMA接管与控制

- DMA控制器向磁盘控制器发起数据传输请求。
- 磁盘控制器将数据从磁盘读入缓冲区。

4. 直接内存传输

- 数据准备好后，磁盘控制器 将数据一个字节一个字节地（或一个字一个字地）通过系统总线传输。
- DMA控制器 管理整个传输过程，在总线上代替CPU扮演主设备的角色，直接将数据写入指定的内存位置。

5. 传输完成与中断

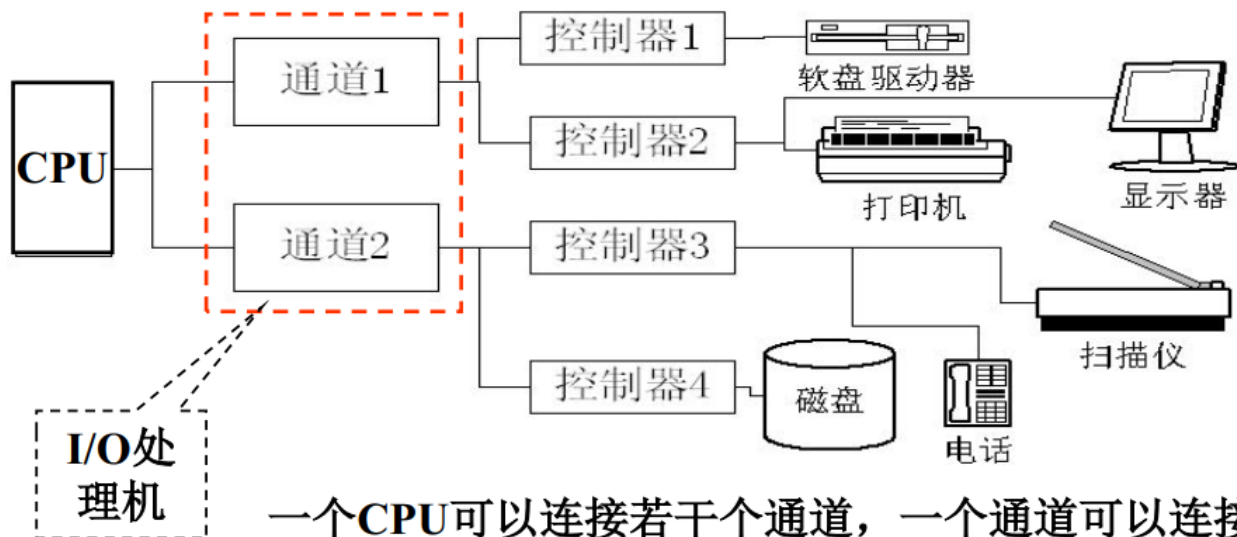
- 当所有数据传输完毕后，DMA控制器向CPU 发送一个中断信号。
- CPU 收到中断，得知I/O操作已完成，随后恢复执行之前被挂起的进程。

流程关键

CPU初始化DMA控制器 → DMA执行具体的数据搬运 → DMA中断通知CPU任务完成

二、I/O系统概述

❖ 通道控制方式I/O



一个CPU可以连接若干个通道，一个通道可以连接若干个控制器，一个控制器可以连接若干个设备。

优点：解决了**I/O操作的独立性**和**各部件工作的并行性**。对比DMA（仅解放数据传输），通道控制让整个**I/O流程与CPU完全并行**：CPU发指令给通道后，通道能独立调度控制器、管理设备，全程不用CPU插手；支持**多通道并行**（通道1管打印机，通道2管磁盘）、**同通道多设备并行**（控制器3连打印机+扫描仪，可同时干活）

缺点：由于**极高的成本和复杂性**，通道控制方式从未在主流的PC和服务器的市场中普及。它被保留在了对I/O性能、可靠性和可扩展性有极致要求的领域——主要是大型机。

二、I/O系统概述

I/O设备的分类

❖ 按数据组织分类

- **块设备(Block Device)**: 指以**数据块**为单位来组织和传送数据信息的设备。它属于**有结构**设备。块设备的基本特征是:
 - ① 传输速率较高, 通常每秒钟为几兆位;
 - ② 它是可寻址的, 即可随机地读/写任意一块;
 - ③ 磁盘设备的I/O采用**DMA**方式。
- **字符设备(Character Device)**: 指以单个字符为单位来传送数据信息的设备。这类设备一般用于数据的输入和输出(交互式终端、打印机)。它属于**无结构**设备。字符设备的基本特征是:
 - ① 传输速率较低;
 - ② 不可寻址, 即不能指定输入时的源地址或输出时的目标地址;
 - ③ 字符设备的I/O常采用中断驱动方式。

二、I/O系统概述

❖ 按数据传输率分类

- **低速设备**：每秒传几个字节~数百字节，**典型设备**：键盘、鼠标、语音输入（比如麦克风录语音）
- **中速设备**：每秒传数千字节~数十千字节，**典型设备**：激光打印机（打印文本/文档，速率适中）
- **高速设备**：每秒传数百千字节~数兆字节，**典型设备**：磁带、磁盘、光盘（存放大文件、批量数据，需要高速读写）

❖ 从资源分配角度分类

- **独占设备**：一段时间只能被一个进程用，**典型设备**：用户终端、打印机（比如打印机打印时，别的进程不能同时用，否则文档会乱）
- **共享设备**：一段时间允许多个进程同时用，**典型设备**：磁盘（多个进程可同时读写不同区域，是文件系统、数据库的基础）
- **虚拟设备**：用虚拟技术把“独占设备”变成“多进程共享的逻辑设备”，靠SPOOLing技术实现（比如把真实打印机虚拟成多个“逻辑打印机”，多用户可同时提交打印任务，后台排队处理）

内容回顾

基本概念

- ❖ 计算机中负责管理I/O的机构称为**I/O系统**（硬件和软件的组合）。

I/O控制方式

- ❖ 直接控制方式
- ❖ 中断驱动方式
- ❖ **DMA**方式
- ❖ 通道控制方式

设备的分类

- ❖ 字符设备/块设备
- ❖ 低速/中速/高速
- ❖ 共享/独占/虚拟

三、I/O软件的组成

I/O软件的组成部分

- **I/O交通管制程序**：当多个I/O设备同时工作时，它负责**协调设备间的“数据通行秩序”**，避免设备冲突、争抢资源（比如打印机和磁盘同时读写，得安排谁先传数据）
- **I/O调度程序**：像“设备管家”，决定哪个进程/任务能优先用I/O设备，**负责设备的分配**（把设备分给谁）和调度（啥时候给、用多久）
- **I/O设备处理程序**：是“设备操作手册”，针对不同类型设备（如磁盘、打印机），写**具体的读写、控制逻辑**（比如磁盘怎么寻道、打印机怎么解析打印指令）

I/O软件设计目标

- **设备独立性**：让用户/程序不用关心具体设备细节（如不管插的是U盘还是移动硬盘，都能用统一方式读写），系统**底层适配不同设备**，上层用通用接口
- **统一命名**：给设备分配**标准化名称**（比如/dev/sda代表某块磁盘），用户/程序通过名字访问设备，不用管物理连接、型号差异

三、I/O软件的组成

I/O软件的结构



用户

用户直接接触的层面（比如办公软件、游戏），通过简单操作（如点击“保存”）发起I/O请求，不用懂设备底层逻辑

用户级软件

系统调用的处理程序——实现“设备独立性”的关键层，把用户级的通用请求，转换成与具体设备无关的指令

与设备无关的系统软件

分层设计思想

设备驱动程序

对接具体设备的“翻译官”，把上层**通用指令**，**转成设备能懂的硬件信号**（比如磁盘驱动把读指令转成**寻道**、**读磁头**操作）

中断处理程序

设备完成I/O操作后，发中断信号，这层**负责“响应中断、善后处理”**（比如数据传完了，通知上层程序继续执行）

外部I/O设备

计算机系统与外界进行数据交互的硬件装置（比如键盘、鼠标、打印机、磁盘、显示器等），负责“物理层面的数据输入/输出”（比如键盘把按键信号传入，显示器把图像信号输出）

用户不能直接操作硬件，需通过I/O软件分层来交互

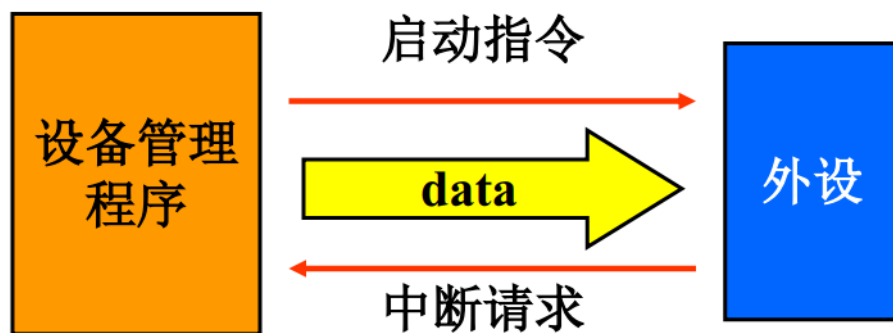
三、I/O软件的组成

中断处理程序

- ❖ 中断机制原理

- ❖ 设置中断的目的：

- 解决高速处理设备和低速输入输出设备之间的矛盾，提高系统工作效率。



三、I/O软件的组成

设备驱动程序

- ❖ 设备驱动程序是直接同硬件打交道的软件模块。一般而言，设备驱动程序的任务为：接受来自与设备无关的上层软件的抽象请求；进行与设备相关的处理。
- ❖ 具体说来，其功能主要有：
 - 控制和监督各I/O控制器的正确执行，并进行必要的错误处理。
 - 处理和设备相关的操作，如排队、挂起、唤醒等。
 - 缓冲区管理。
 - 进行一些较为底层的和具体设备相关的处理工作。



设备驱动程序处理的都是和底层硬件设备紧密相关的操作和错误处理。

三、I/O软件的组成

❖ 设备驱动程序的特点

- 与I/O设备的硬件结构密切联系，是操作系统底层中唯一知道各种输入输出设备的控制器细节及其用途的部分。

例如，只有磁盘驱动程序具体了解磁盘的区段、磁道、柱面、磁头、磁臂的运动、交错访问系数、马达驱动器、磁头定位次数以及所有保证磁盘正常工作的机制，其他软件根本不过问这些硬件操作的细节。

❖ 设备驱动程序的结构

- 由于驱动程序和硬件的结构有着密切的联系，因此不同的硬件其启动程序的结构以也不同。
- 但是对于略有差异的同一类设备，为了方面使用，系统往往会提供一个通用的设备驱动程序。当然为了追求更好的性能，用户可以使用厂家提供的专门为该设备编写的设备驱动程序。

三、I/O软件的组成

与设备无关的系统软件

- ❖ 是建立在设备驱动程序之上的，与具体设备无关的I/O功能的集合(例如所有设备都需要的I/O功能)。
- ❖ 功能：
 - **统一命名**：将设备的符号名映射到相应的设备驱动程序上，对外提供同一的命名方式。
 - **设备保护**：对设备进行必要的保护，防止无授权的应用或用户的非法使用。
 - **提供与设备无关的逻辑块**：屏蔽底层各种I/O设备空间大小、处理速度和传输速率的差异，只向上层提供大小统一的逻辑块尺寸。
 - **缓冲管理**
 - **存储设备的块分配**：查找一个存储设备的空闲块并进行分配。
 - **独占设备的分配和释放**
 - **出错处理**：一般来说I/O错误有两种
 - ① 操作故障：由驱动程序处理。
 - ② 非操作故障：如磁盘受损而不能再读，由与设备无关的系统软件处理，并向上层返回出错信息。

三、I/O软件的组成

与设备无关的系统软件——举例

功能	解释	举例
统一命名	提供统一的访问入口，隐藏设备实际地址。	在Linux中，可能被统一映射为 /dev/sda 或通过文件系统路径如 /home/user/file.txt 来访问。
设备保护	检查用户或程序是否有权使用设备。	当用户尝试写入一个USB设备时，系统会检查是否有写权限，而不是直接把控制权交给驱动程序。
提供统一逻辑块	屏蔽物理设备扇区大小的差异。	硬盘物理扇区可能是512字节或4096字节，但操作系统向上层文件系统提供统一的“块”（如4KB）作为读写单位。
缓冲管理	在内存中建立缓存，平滑速度差异。	read 操作可能直接从内存缓冲区返回数据，而不是每次都去读慢速的磁盘。
块分配	管理磁盘的空闲空间。	文件系统（如NTFS, ext4）负责记录哪些块是空闲的，并在创建文件时进行分配。这层逻辑与具体的硬盘型号无关。
独占设备分配	管理不能共享的设备。	当一台打印机正在被一个进程使用时，系统会拒绝其他进程的打印请求，并将其加入等待队列。
出错处理	处理通用的、非设备特有的错误。	如果磁盘某个扇区物理损坏（非操作故障），与设备无关的软件会向上层返回一个“I/O错误”，而不是让驱动程序特有的错误码直接暴露给应用程序。

三、I/O软件的组成

用户空间的I/O软件

❖ 常见的主要有

- **I/O系统调用**：用户程序通过“系统调用”（比如open、read、write）发起I/O请求，不用自己写复杂驱动逻辑
- **Spooling系统**：把“独占设备”（比如打印机）变成“虚拟共享设备”。多个用户同时提交打印任务，Spooling先存到磁盘队列，再排队打印，感觉像“同时在用打印机”，提升设备利用率

三、I/O软件的组成

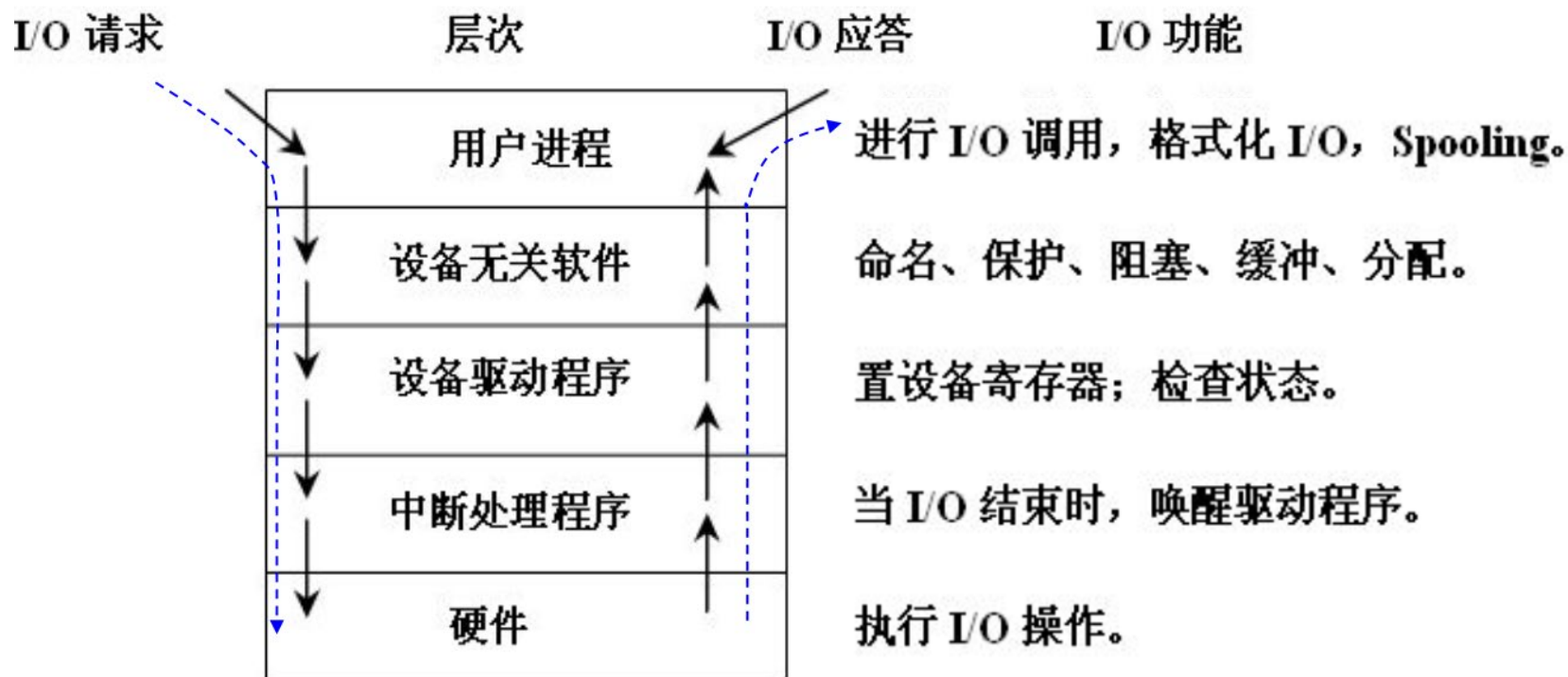
总结——I/O软件的“分层协作”：

- 最底层是**中断处理程序**，负责“设备完成通知”；
- 贴近硬件的是**设备驱动程序**，负责“硬件操作翻译”；
- 中间层是**与设备无关的系统软件**，负责“通用逻辑+屏蔽差异”；
- 最上层是**用户空间的I/O软件**，负责“让用户/程序简单用I/O”。

每层各司其职，把复杂的I/O流程拆成简单步骤，既让硬件能被灵活控制，又让用户/程序用起来方便。

三、I/O软件的组成

I/O系统的层次结构与每层的主要功能



三、I/O软件的组成

现代操作系统的I/O处理流程——（以“打印文档”为例）

1. 用户请求（异步非阻塞——应用无需等待）

- Word通过系统调用将打印任务提交给操作系统假脱机服务。提交后Word立即恢复响应，用户可继续工作。

2. 设备无关处理（调度与转换）

- **假脱机服务（Spooling）** 接管任务，执行两大核心操作：
 - **格式转换**：调用驱动，将文档转换为打印机专用指令（如PDF）。
 - **任务排队**：将转换后的数据暂存至磁盘，进入打印队列等待。

3. 驱动执行（输出至硬件）

- 功能驱动：将打印指令转换为打印机硬件语言（如PCL）。
- 总线驱动：通过USB、网络等将数据和指令发送至物理打印机。

4. 硬件与中断（异步响应）

- 打印机接收数据并执行打印；完成后，通过硬件中断通知CPU；中断处理程序读取状态，并向上层驱动反馈。

5. 异步通知（完成）

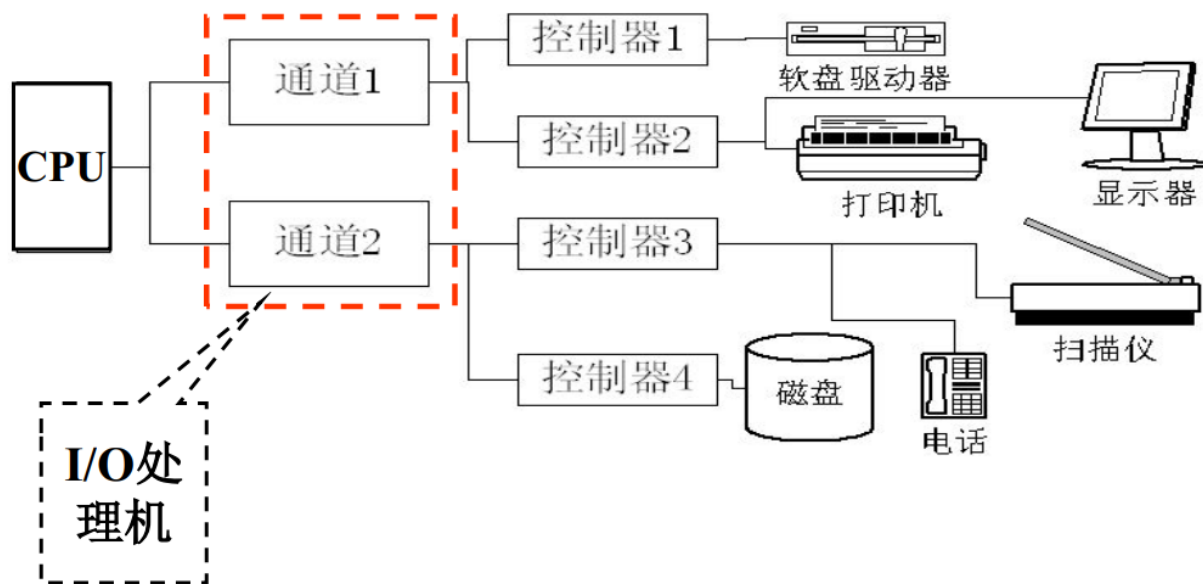
- 驱动将完成状态反馈给假脱机服务；假脱机服务清除队列中的任务，并异步通知Word；Word向用户显示“打印完成”提示。

四、具有通道的设备管理

通道技术

◆ **通道**：是CPU与外设间的“专用I/O处理机”，能独立管理设备I/O，让CPU彻底解放。

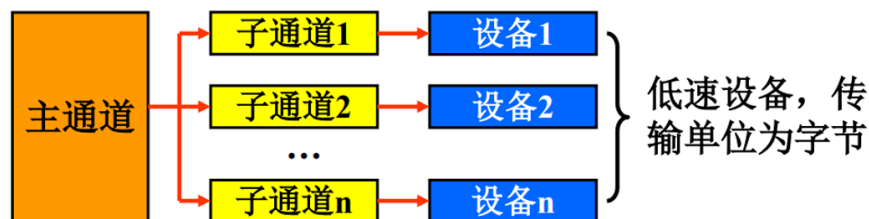
CPU连多个通道（如通道1、通道2），每个通道连多个控制器，每个控制器连多个外设，形成“CPU→通道→控制器→设备”的分层管理。



四、具有通道的设备管理

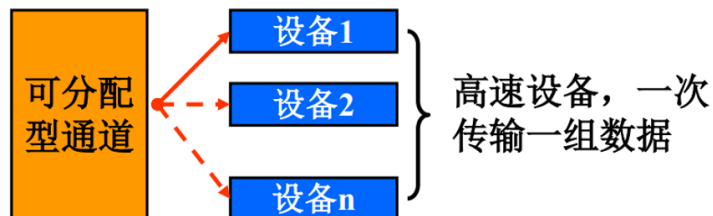
通道的三种类型：

- **字节多路通道** (Byte Multiplexor Channel)：管理多个低速/中速设备（如键盘、鼠标、打印机），主通道拆成多个“子通道”，轮流为多个低速/中速设备服务。**按字节为单位交叉传输数据**，适合多设备并发的场景。**比如**：子通道1给键盘传1个字节→子通道2给鼠标传1个字节→子通道3给打印机传1个字节...循环往复。



优点：“不浪费通道资源”，低速设备传输慢，通道可以分时复用，提高整体利用率

- **数组选择通道** (Block Selector Channel)：管理单个高速设备（如磁盘、磁带），通道是“可分配型”，专为高速设备设计，一次独占整个通道，**按数据块为单位传输**。**比如**：给磁盘发指令，通道全程专属磁盘，直到数据块传输完成。



优点：高速设备需要连续、大量传输时，避免被其他设备打断，保证传输效率

四、具有通道的设备管理

- **数组多路通道** (Block Multiplexor Channel)：结合前两者优点，既支持高速设备按块传输，又能分时为多个设备服务，平衡“效率”和“并发”

优点：平衡“传输速度”和“通道利用率”，适合既有高速设备、又要多设备并发的场景（比如服务器同时读写磁盘、处理网络I/O）

◆ 三种通道通过总线与CPU、内存相连，并行独立地工作：

分工：CPU不再直接处理I/O细节，而是根据I/O请求的类型（低速字符、中速数据块、高速批量数据），将任务“派发”给相应的通道。字节多路通道处理“琐碎事”，数组多路通道处理“并行事”，数据选择通道处理“大事”。

并行：当数据选择通道将一个大文件从磁盘读入内存时，字节多路通道可以同时处理用户的键盘输入，数组多路通道也可以同时操作多个磁带机。

这种协作最终实现了**负载均衡**（不同速度负载被分流到专用通道）和**资源高效利用**（CPU被解放，各类I/O设备都能以接近自身最高效率的方式工作）。

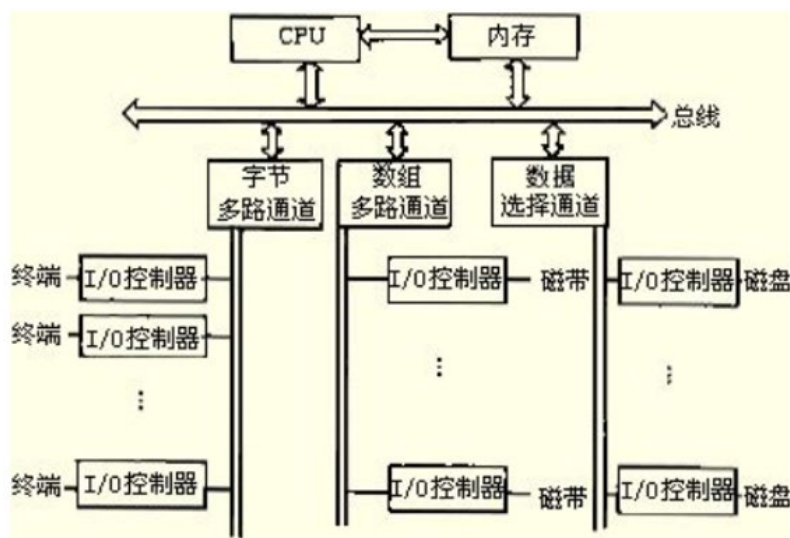
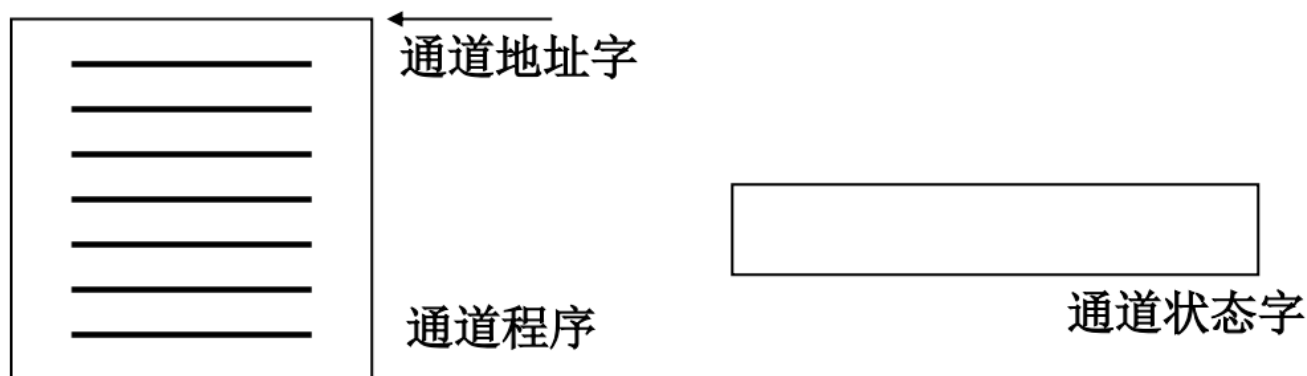


图 5-6 通道方式的数据传送结构

四、具有通道的设备管理

术语：

- ❖ **通道命令(Channel Command Word, CCW)**：通道又称为I/O处理机，具有自己的指令系统，常常把I/O处理机的指令称通道命令。
- ❖ **通道程序**：用通道命令编写的程序称通道程序，通道通过执行通道程序控制I/O设备运行。
- ❖ **通道地址字(Channel Address Word, CAW)**：用来存放通道程序首地址的内存单元称通道地址字。
- ❖ **通道状态字(Channel Status Word, CSW)**：是通道向操作系统报告工作情况的状态汇集。

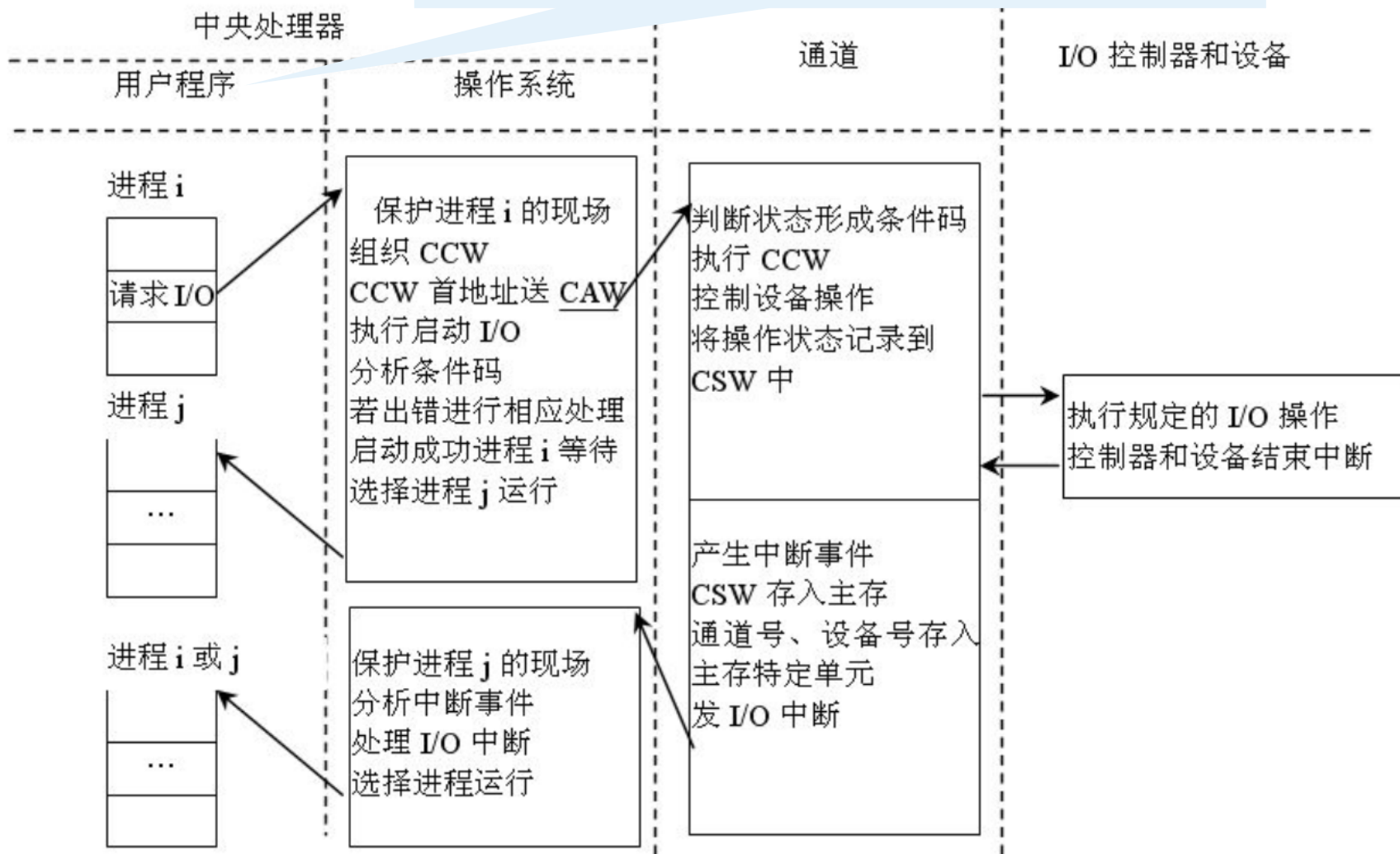


四、具有通道的设备管理

通道的工作原理

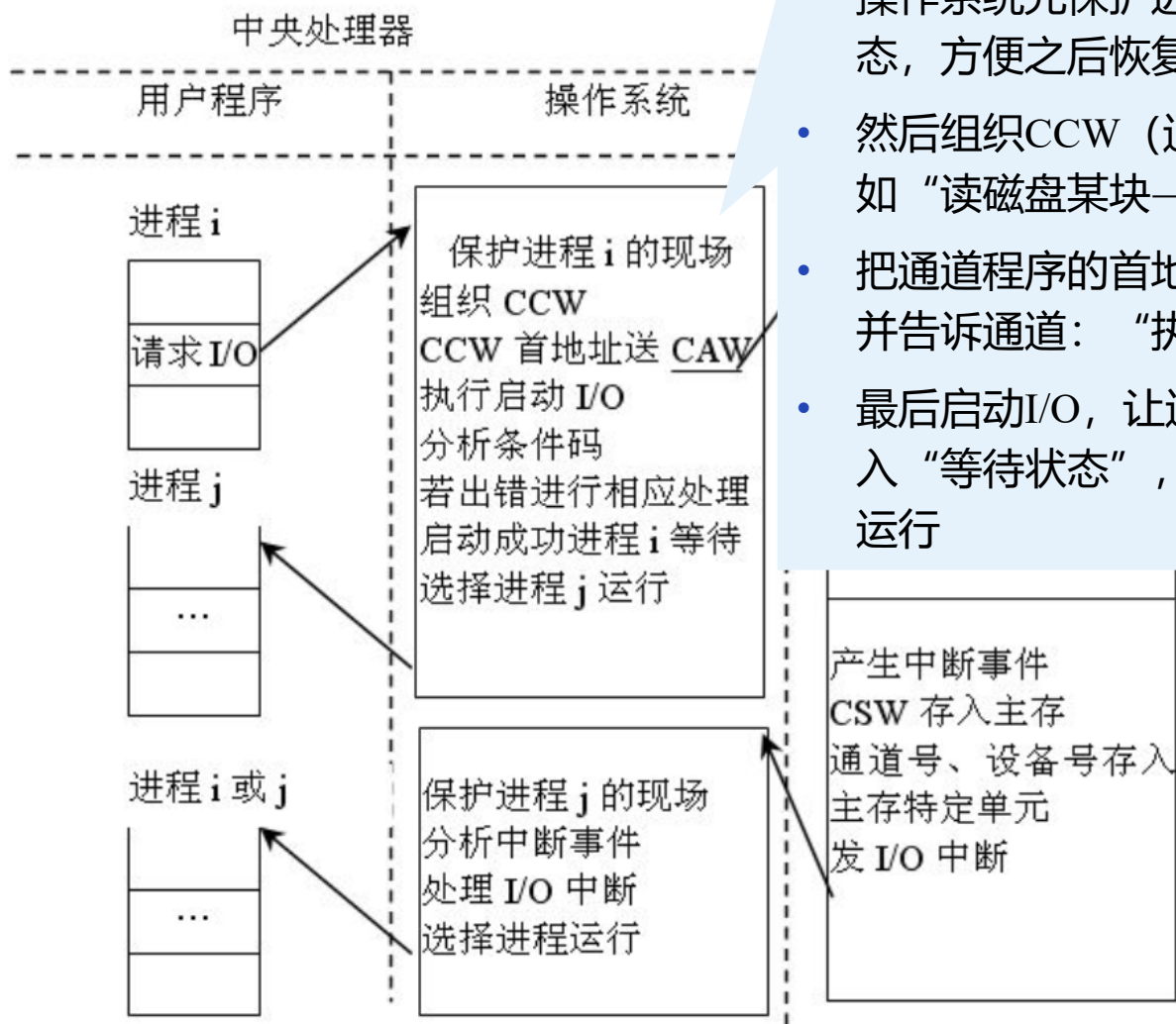
阶段1：用户程序发起I/O请求（以进程 i 为例）

进程 i：“要读写数据”（比如打开文件、保存数据）



四、具有通道的设备管理

通道的工作原理

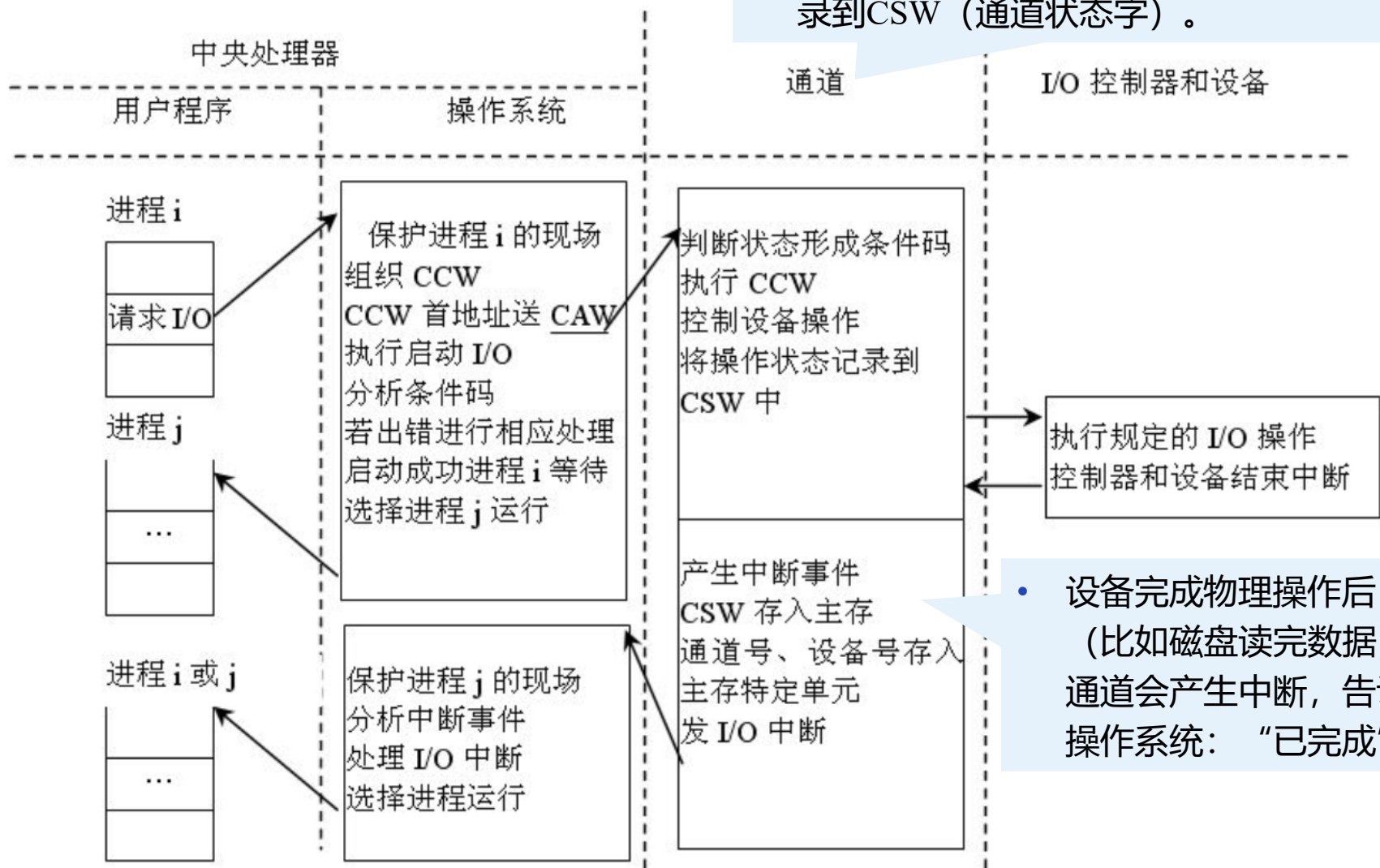


阶段2：操作系统“组织通道干活”

- 操作系统先保护进程 i 的现场（记录进程当前状态，方便之后恢复）；
- 然后组织CCW（通道命令），编写通道程序（比如“读磁盘某块→存到内存某地址”）；
- 把通道程序的首地址存到CAW（通道地址字），并告诉通道：“执行这个程序”；
- 最后启动I/O，让通道开始工作，同时让进程 i 进入“等待状态”，去调度其他进程（比如进程 j）运行

四、具有通道的设备管理

通道的工作原理



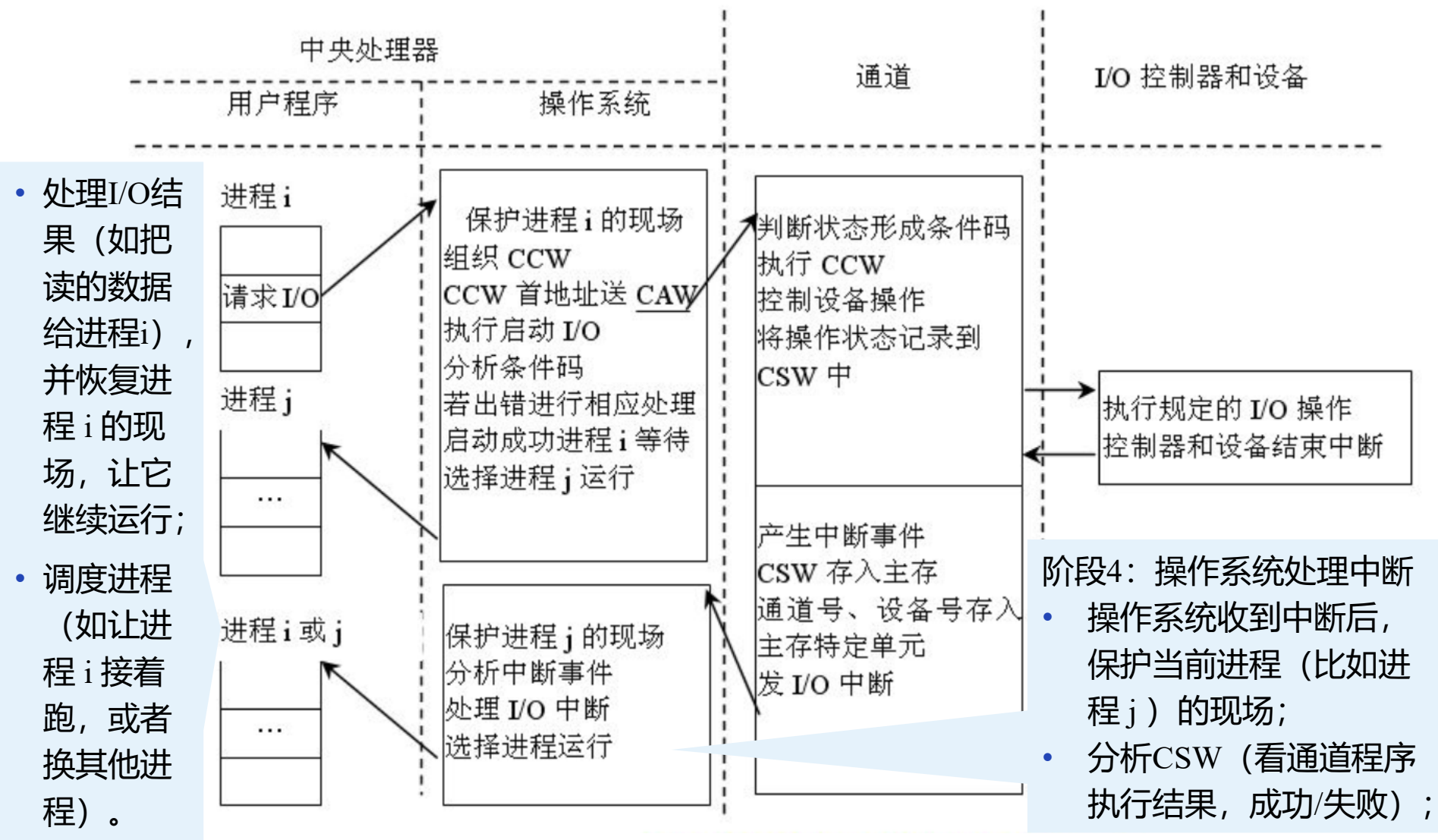
阶段3：通道执行I/O操作

- 通道从CAW找到通道程序，逐行执行CCW（比如发指令给磁盘控制器，让它读数据）；
- 执行过程中，把状态（成功/失败、进度）记录到CSW（通道状态字）。

- 设备完成物理操作后（比如磁盘读完数据），通道会产生中断，告诉操作系统：“已完成”

四、具有通道的设备管理

通道的工作原理



四、具有通道的设备管理

通道技术的现代定位

我们常用的笔记本电脑和现代通用操作系统（如Windows, Linux, macOS）在其标准架构中，并不使用传统大型机意义上的“通道”技术。

通道的思想已被“吸收”和“演化”。虽然不用传统通道，但通道的核心思想“**将I/O管理任务从CPU卸载到专用硬件**”已经深深融入现代PC架构中，并以更经济、更通用的方式实现：

特性	传统通道技术	现代PC/笔记本的I/O方案
核心硬件	专用通道处理器	DMA控制器 + 多功能芯片组
成本	极高	低，高度集成
复杂度	高，需要专门的操作系统支持	相对较低，已标准化
应用场景	IBM大型机等关键任务服务器	通用计算、个人电脑、普通服务器
哲学体现	“专用硬件解决专用问题”	“通用硬件+标准化接口”

五、缓冲技术



缓冲技术

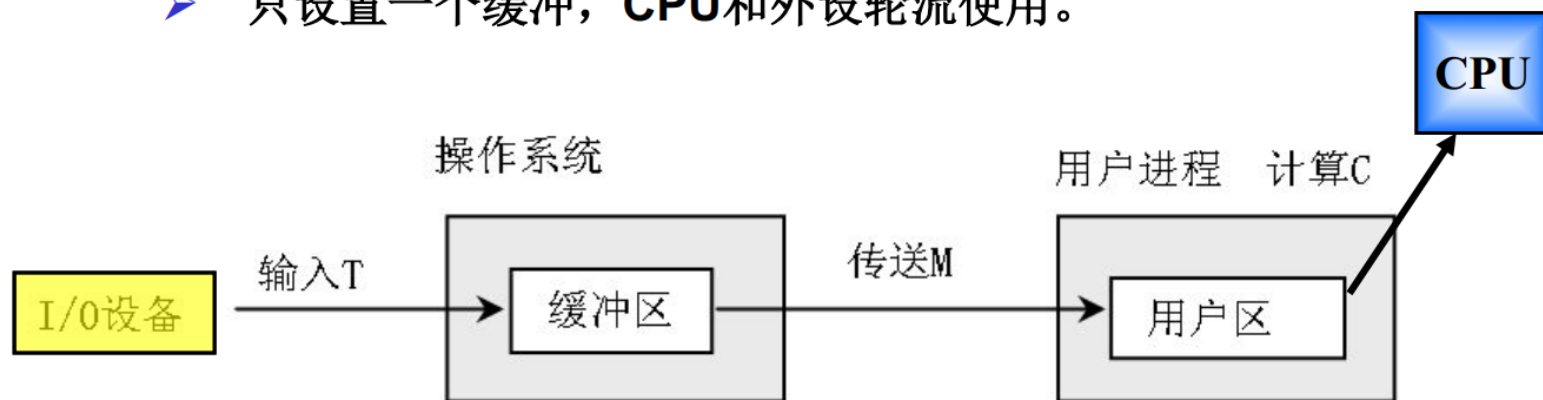
- ❖ 缓冲区是一种交换数据的区域。



缓冲技术的分类

- ❖ 单缓冲技术(single buffer)

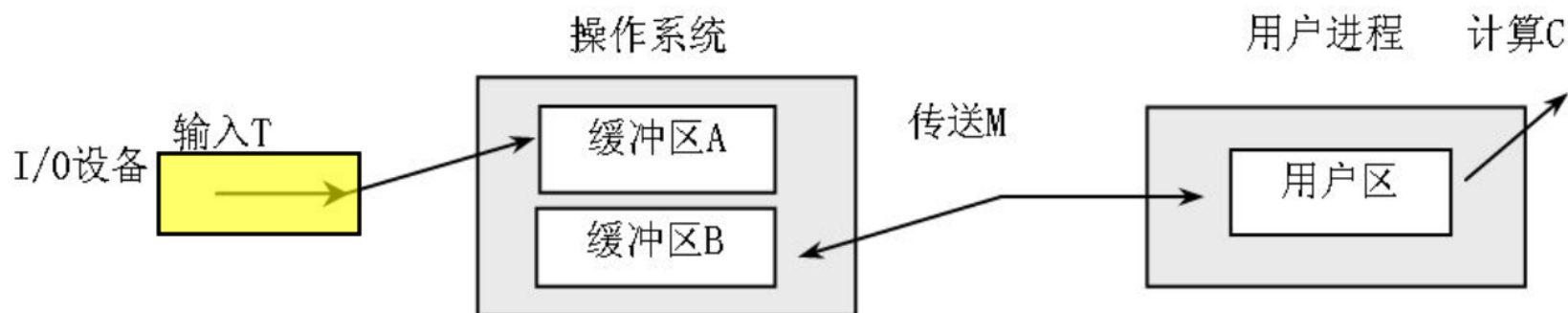
- 只设置一个缓冲，CPU和外设轮流使用。



计算过程和数据输入缓冲的过程可以并行。

五、缓冲技术

❖ 双缓冲(double buffer)



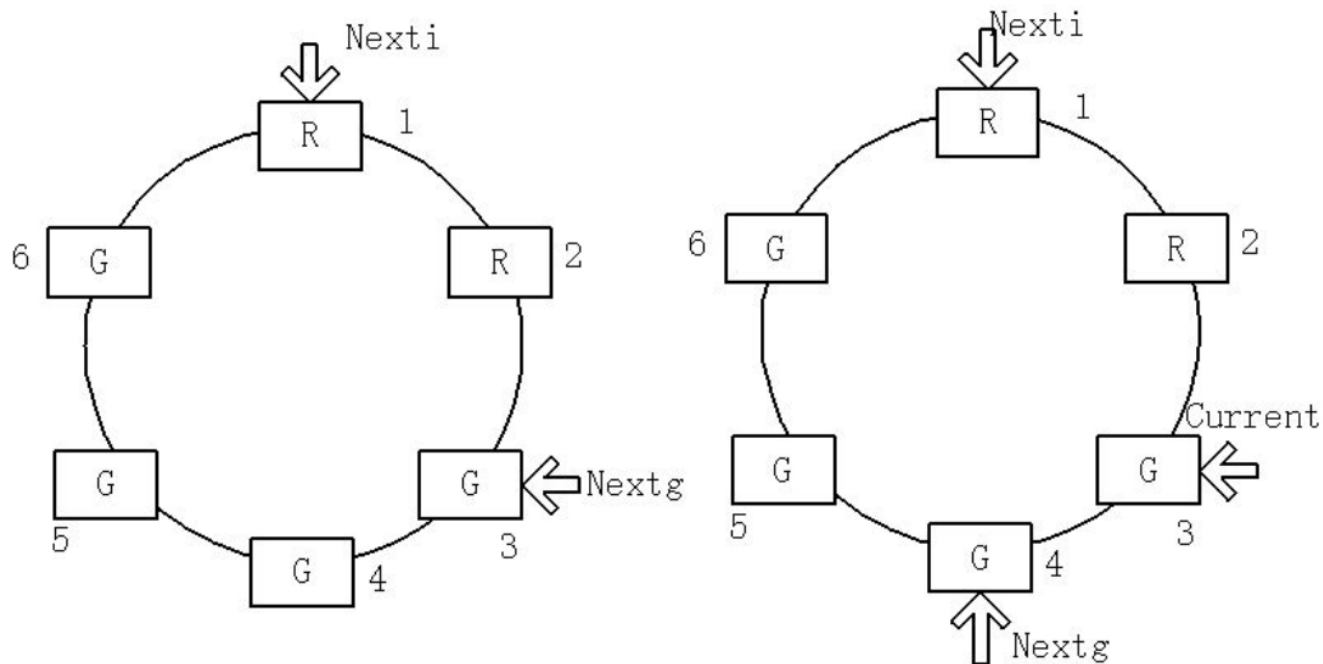
计算过程、数据输入缓冲的过程、数据传送的过程可以并行。

适合于外设速度较高的情况。

五、缓冲技术

❖ 环形缓冲

➤ 结构:



把缓冲区排成一个“圈”，数据像“接力赛”一样在圈里传，**圈里有两种缓冲区：**

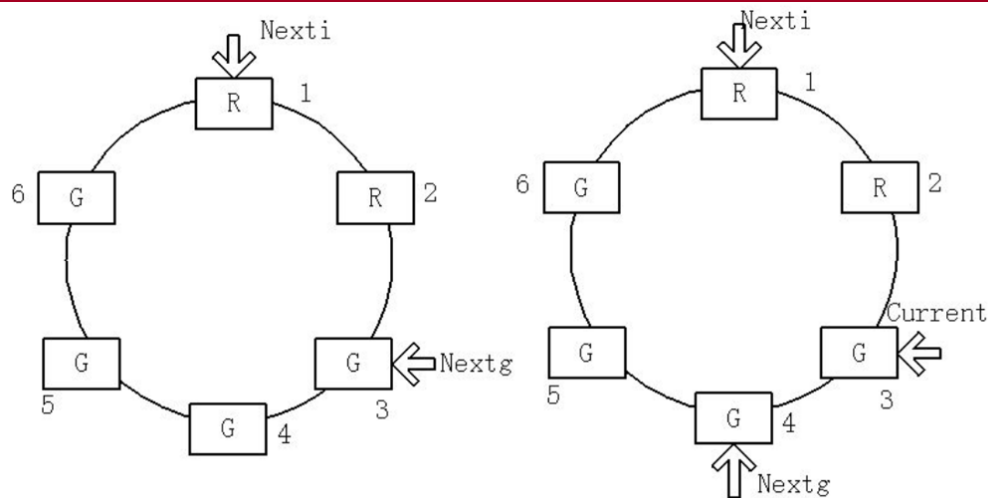
- **R（接收区）**：外设传数据进来的地方，表示数据已就绪，可以被读取。
- **G（释放区）**：CPU取走数据的地方，表示该位置为空闲，可以写入新数据。

五、缓冲技术

环形缓冲的两种现象

系统受限计算: **Nexti**追上**Nextg**

系统受限I/O: **Nextg**追上**Nexti**

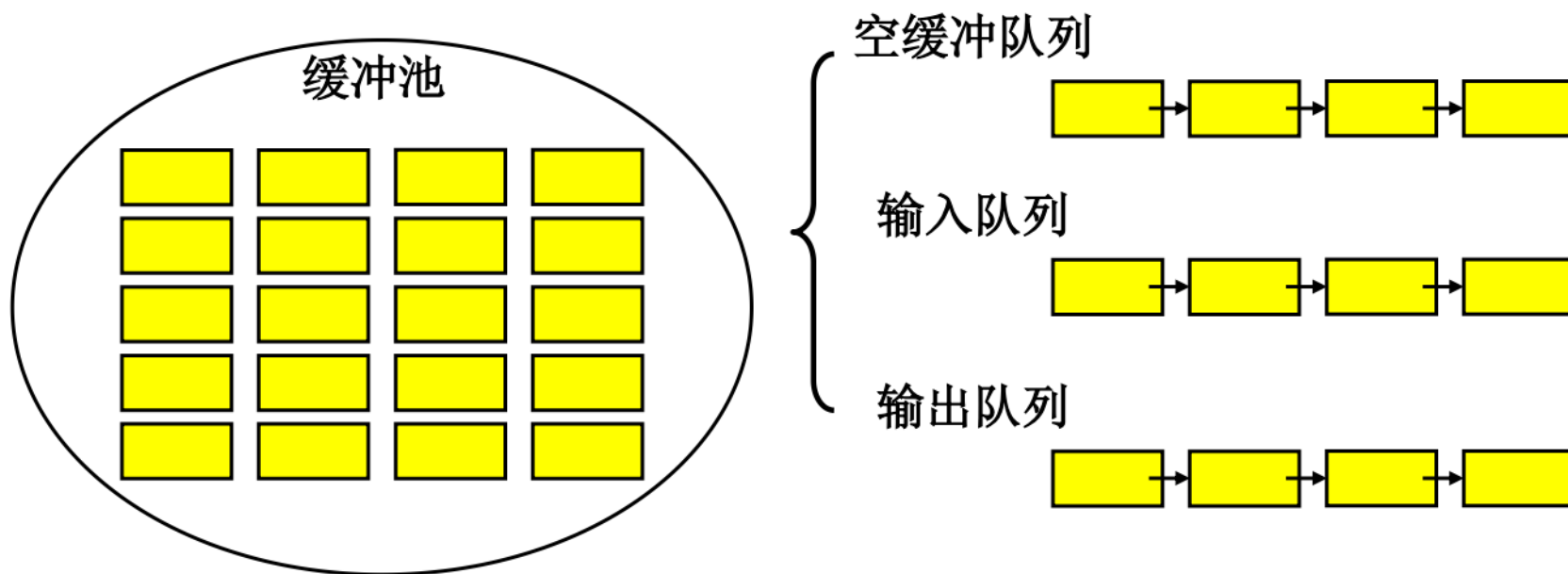


- **Nexti**: 表示下一个可用于接收数据的缓冲区位置，其中“i”代表“input（输入）”。在数据输入过程中，外设会将数据写入Nexti所指向的缓冲区。每完成一次数据写入操作，Nexti就会按照环形缓冲的顺序移动到下一个缓冲区位置，以便为下一次数据输入做好准备。比如，在一个包含多个缓冲区单元的环形缓冲结构中，Nexti会依次指向各个空闲的缓冲区，引导数据按序存入
 - **Nextg**: 表示下一个可用于释放（供CPU读取并处理）数据的缓冲区位置，其中“g”代表“get（获取）”。当CPU需要从环形缓冲中读取数据进行处理时，会从Nextg所指向的缓冲区中获取数据。同样，每完成一次数据读取操作，Nextg也会按照环形缓冲的顺序移动到下一个缓冲区位置，确保数据能够被按顺序处理
- 当**Nexti**追上**Nextg**时，意味着输入速度过快，缓冲区已满，系统计算受限；
 - 当**Nextg**追上**Nexti**时，则表示输出速度过快，缓冲区已空，系统I/O受限。

五、缓冲技术

❖ 缓冲池(buffer pool)

- 可供多个进程共享的双向缓冲技术。



小结

